

Project Report: Unified Explainable AI Interface

Course: Explainability AI - ESILV A5

Templé Valentin, Ranc Paul, Rech Valentin

Janvier 2026



Table des matières

1	Introduction and Context	3
1.1	Project Objective	3
1.2	Use Cases	3
1.3	Problem Statement	3
2	Technical Architecture	3
2.1	Project Structure	3
2.2	Technologies Used	3
2.3	Classification Models	4
3	Explainability Techniques (XAI)	4
3.1	LIME (Local Interpretable Model-agnostic Explanations)	4
3.2	Grad-CAM (Gradient-weighted Class Activation Mapping)	4
3.3	SHAP (SHapley Additive exPlanations)	4
4	XAI Methods Comparison	5
4.1	Performance Analysis	5
4.2	Complementarity of Approaches	5
5	Conclusion and Perspectives	5
5.1	Limitations and Future Improvements	5
6	Generative AI Usage Statement	5

1 Introduction and Context

1.1 Project Objective

This project aims to develop a unified **Explainable Artificial Intelligence (XAI)** interface capable of processing and analyzing two different data types : **audio and image**. The main objective is to demonstrate the importance of explainability in deep learning systems by integrating model interpretation techniques to make predictions transparent and understandable.

1.2 Use Cases

The project integrates two distinct detection systems :

- **Deepfake Audio Detection** : Identification of authentic vs. falsified audio recordings.
- **Lung Cancer Detection** : Analysis of chest X-rays to detect malignant tumors.

1.3 Problem Statement

In critical domains such as healthcare and cybersecurity, it is essential to understand why a deep learning model makes a particular decision. This project addresses this issue by implementing several XAI techniques that enable visualization and interpretation of model decisions.

2 Technical Architecture

2.1 Project Structure

The architecture follows a modular design to promote maintainability and extensibility.

Key advantages :

- Separation of concerns (UI, business logic, data processing).
- Component reusability.
- Easy addition of new models or XAI techniques.

2.2 Technologies Used

- **UI Framework** : Streamlit (interactive and responsive interface).
- **Deep Learning** : TensorFlow/Keras (classification models).
- **XAI** : LIME, Grad-CAM, SHAP (explainability techniques).
- **Data Processing** :
 - Audio : Librosa (Mel-spectrogram generation).
 - Image : OpenCV, scikit-image (preprocessing).

2.3 Classification Models

Audio (Deepfake Detection) : Modular architecture with three selectable models :

- **Custom_PB** : Custom trained model (TensorFlow SavedModel)
- **MobileNetV2** : Pre-trained ImageNet + custom head (2 classes)
- **ResNet50** : Pre-trained ImageNet + custom head (2 classes)
- **Input** : Mel-spectrogram ($224 \times 224 \times 3$) generated via `librosa`
- **Architecture** : GlobalAveragePooling2D $\rightarrow$ Dense(2, softmax)
- **Output** : Real vs. Fake probability (binary classification)

Image (Lung Cancer Detection) : Primary model : `best_chest_xray_model.h5` (custom trained model). Alternative models : DenseNet121 and AlexNet (ImageNet weights, demonstration mode).

- **Input** : Chest X-ray ($224 \times 224 \times 3$)
- **Output** : Binary classification Normal vs. Pneumonia/Abnormal
- **Specificity** : Loading with `compile=False` for compatibility

3 Explainability Techniques (XAI)

3.1 LIME (Local Interpretable Model-agnostic Explanations)

Principle : Locally approximates the model's behavior by perturbing the input and observing the impact on predictions.

1. Image segmentation into super-pixels.
2. Generation of variations by masking certain segments.
3. Training a local linear model.
4. Identification of the most influential regions.

3.2 Grad-CAM (Gradient-weighted Class Activation Mapping)

Principle : Uses gradients from the final convolutional layer to generate a heatmap highlighting important regions.

1. Computation of gradients of the predicted class with respect to feature maps.
2. Weighting of feature maps by these gradients.
3. Generation of a heatmap overlaid on the original image.

3.3 SHAP (SHapley Additive exPlanations)

Principle : Based on game theory, SHAP calculates the contribution of each feature to the prediction.

1. Application of KernelExplainer for complex models.
2. Computation of Shapley values for each pixel/segment.
3. Visualization of positive and negative contributions.

4 XAI Methods Comparison

4.1 Performance Analysis

TABLE 1 – Comparison of XAI Methods

Method	Strength	Weakness
LIME	Excellent local identification	Sensitive to sample count
Grad-CAM	Precise and fast (Heatmaps)	Model-specific (ConvNets)
SHAP	Most complete (Game Theory)	High computation time

4.2 Complementarity of Approaches

The combined use of these three techniques offers :

- Cross-validation of important areas.
- Detection of potential model biases.
- Increased confidence in critical predictions.

5 Conclusion and Perspectives

5.1 Limitations and Future Improvements

- **Model Training** : Some models, particularly for the audio component, did not benefit from full or optimal training on the specific voice dataset, which may impact prediction accuracy.
- **Performance** : High computational overhead for SHAP when processing high-resolution images.
- **Compatibility** : Limited Grad-CAM compatibility with specific model formats or non-convolutional architectures.

6 Generative AI Usage Statement

In accordance with the project guidelines, we declare the use of Generative AI tools during the research and development phases of this project :

- **Perplexity AI** : Used as a specialized research tool to locate pre-trained weights and models compatible with the **CheXNet dataset**. This step was necessary due to the absence of pre-trained models in the original project repository, ensuring the medical imaging component of our interface remains robust and functional.
- **Gemini 3** : Utilized as a primary coding assistant. It helped in architecting the unified Streamlit interface, debugging complex interactions between TensorFlow models and XAI libraries (notably SHAP and Grad-CAM), and optimizing the data processing pipelines for both audio and image inputs.